

RecursiveMAS: What Happens When AI Agents Stop Talking and Start Sharing Latent Thoughts?

A practical breakdown of “Recursive Multi-Agent Systems,” a new framework that tries to make multi-agent AI faster, cheaper, and better by moving collaboration from text tokens into latent space.



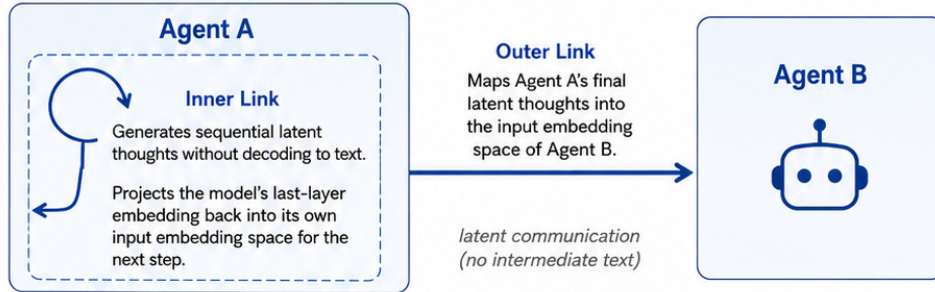
AI ENGINEERING INSIDER

MAY 10, 2026

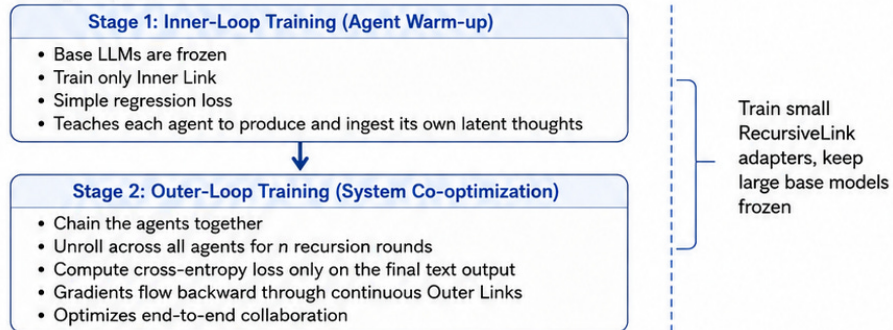
How RecursiveMAS Works

A latent-communication multi-agent framework trained in two stages

1. The Architecture



2. Training Pipeline



3. Why This Matters for Production



Drastic Cost Reductions

Eliminates repeated input/output token costs between agents.



Ultra-Low Latency

Removes autoregressive decoding bottlenecks for real-time multi-agent systems.



End-to-End Fine-Tuning

Optimize how agents collaborate by training only lightweight adapters (~13M params).



Hardware Efficiency

Requires much less peak VRAM than full fine-tuning or LoRA.

4. Implementation Notes



Infrastructure

Requires direct access to model weights. Black-box APIs are not enough. Likely built with Hugging Face Transformers or customized vLLM.



The Adapter

RecursiveLink is a lightweight PyTorch adapter that projects hidden states across embedding spaces.

$h_t \rightarrow \text{RecursiveLink} \rightarrow e_{t+1}$



Memory Management & Data

Track continuous embeddings instead of token IDs. Custom KV-cache handling may be needed. Training can use standard input/output pairs—no intermediate text traces required.



Key Idea: RecursiveMAS lets agents think and communicate in latent space, making multi-agent AI faster, cheaper, and more trainable.

The Problem This Paper Is Solving

In standard multi-agent systems (such as AutoGen or CrewAI), agents interact sequentially via text.

- **The Latency Problem:** Agent 2 cannot start thinking until Agent 1 finishes printing its final token.
- **The Cost Problem:** You pay for the output tokens of Agent 1, and then you pay again for those exact same tokens as the input context for Agent 2.
- **The Optimization Problem:** If Agent 3 makes a mistake, how do you update Agent 1's parameters to prevent it? You can't easily backpropagate gradients through discrete text tokens (the "gradient vanishing" problem).

Current solutions rely on prompt engineering or training agents in isolation. But updating agents independently doesn't optimize the *collaboration* between them.

This Substack is reader-supported. To receive new posts and support my work, consider becoming a free or paid subscriber.

✓ Subscribed

How the Method Works

The RecursiveMAS framework operates in a loop and is trained in two distinct stages.

The Architecture

- **Inner Link:** Allows a single agent to generate sequential latent thoughts without decoding to text. It takes the model's last-layer embedding and projects it back into the model's own input embedding space for the next step.

- **Outer Link:** Bridges the gap between *different* agents. It maps Agent A's final latent thoughts into the input embedding space of Agent B.

The Training Pipeline

1. **Stage 1: Inner-Loop Training (Agent Warm-up).** The base LLMs are completely frozen. The system trains only Inner Link for each agent using a simple regression loss. This teaches the agent how to reliably produce and ingest its own latent thoughts.
2. **Stage 2: Outer-Loop Training (System Co-optimization).** The agents are chained together. The system unrolls the forward pass across all agents for n recursion rounds. A cross-entropy loss is calculated only on the final text output. Gradients flow backward through the continuous Outer Links, optimizing end-to-end collaboration across the entire system.

Why This Matters for Production AI Systems

For applied AI engineers and architects, RecursiveMAS introduces a paradigm shift in how we build autonomous systems:

1. **Drastic Cost Reductions:** If you run heavy multi-agent orchestrations (e.g., coding assistants that plan, write, and critique), switching to latent communication eliminates the redundant input/output token costs of intermediate steps.
2. **Ultra-Low Latency Agents:** Latent transfer completely removes the auto-regressive decoding bottleneck between agents. This paves the way for real-time, multi-agent voice or video systems where text-decoding latency is a dealbreaker.
3. **True End-to-End Fine-Tuning:** Until now, fine-tuning a multi-agent system meant fine-tuning each agent independently. RecursiveMAS allows you to freeze the massive base models and train only the

tiny RecursiveLink adapters (around 13M parameters). You can finally optimize how agents talk to each other based on the final system outcome.

4. **Hardware Efficiency:** RecursiveMAS requires significantly less peak VRAM than full supervised fine-tuning or LoRA training because only the lightweight links are updated.

Implementation Notes for AI Engineers

If you want to build this, be prepared to step away from standard API calls.

- **Infrastructure:** You cannot use black-box APIs (like OpenAI or Anthropic) for this. You need direct access to the model weights to extract and inject last-layer hidden states. You will likely need to customize inference engines like vLLM or Hugging Face Transformers.
- **The Adapter:** This RecursiveLink is trivial to implement in PyTorch. It's just:

$$h_{out} = W_3 h_{in} + W_2 \text{GELU}(W_1 h_{in})$$

- **Memory Management:** Passing continuous embeddings requires tracking and caching continuous vectors rather than discrete integer token IDs. This may require custom KV-cache modifications depending on your serving infrastructure.
- **Training Data:** You will need standard input/output pairs (e.g., question and final answer). The beauty is you don't need to generate intermediate text traces for supervision—the outer loop uses Cross-Entropy on the final output.

Paper link: <https://arxiv.org/abs/2604.25917>

This Substack is reader-supported. To receive new posts and support my work, consider becoming a free or paid subscriber.

✓ **Subscribed**